

Assignment 5 — Timeseries or Text

DATA504 · Due end of Week 10

Choose **one** track. Delete or leave unused the other section. Less scaffolding is intentional: design choices are part of the work.

Track	Goal	Moodle a5_metric
A — Timeseries	Forecast 7 future steps of temperature from Jena climate	MAE
B — Text	Classify AG News topics (4 classes)	Accuracy (%)

Tasks (whichever track)

- 1. Prepare inputs (windowing **or** tokenisation / vectorisation)
- 2. Train a sequence model appropriate to the track
- 3. Report the metric above
- 4. Short written answers: architecture + how inputs were prepared

Moodle fields

a5_repo , a5_track , a5_metric , a5_architecture , a5_tokenisation

AI disclosure (required)

Fill the table in the next cell before you submit.

AI disclosure

Field	Your answer
Tool / model	
Used for	
Verified how	
Not used for	

In [7]:

```
TRACK = "B" # "A" or "B" — Moodle a5_track

import numpy as np
import matplotlib.pyplot as plt
import keras
from keras import layers

print("Selected track:", TRACK)

Selected track: B
```

Track A — Timeseries (Jena)

Forecast the next 7 steps of T (degC) from a past window of observations.

You decide: lookback length, normalisation, train/val/test split, LSTM vs Conv1D vs other, and how you compute MAE on the original °C scale.

```
In [5]: if TRACK == "A":
import pandas as pd
import tensorflow as tf
from pathlib import Path

# 1. obtain Jena climate CSV (download / unzip as needed)
uri = "https://storage.googleapis.com/tensorflow/tf-keras-datasets/jena_climate_2009_2016.csv.zip"
zip_path = keras.utils.get_file(
    origin=uri,
    fname="jena_climate_2009_2016.csv.zip",
    extract=True,
) #download from internet, filename, 是否解压
extract_dir = Path(zip_path).with_suffix("") #去掉.zip后缀
csv_path = next(extract_dir.glob("*.csv")) #文件夹里找到csv文件
df = pd.read_csv(csv_path)
print(df.shape)
print(df.head())

# TODO: select the temperature series (and optionally other covariates)
# 2. Prepare features and target
feature_df = df.drop(columns=["Date Time"]) # 删掉时间字符串列, 时间文本不能直接喂神经网络
feature_names = feature_df.columns.tolist() #把特征列名转成列表, 方便后面查找温度是第几列;

temperature = df["T (degC)"].astype("float32").to_numpy() #单独提取温度列 T (degC), 转成 float32 的 numpy 数组, 这就是我们要预测的目标 y;
features = feature_df.astype("float32").to_numpy() # 所有气象特征转成 numpy 数组, 作为模型输入 X;

temp_col = feature_names.index("T (degC)") #找到温度这一列在特征矩阵里的索引, 后面朴素基准模型需要用这个位置拿温度值。

# 3. Chronological train / validation / test split 时序数据绝对不能随机打乱划分, 必须按时间先后切分!
n = len(features)
num_train = int(0.5 * n)
num_val = int(0.25 * n)
num_test = n - num_train - num_val

print(num_train, num_val, num_test)

# TODO: normalise using statistics from the *training* portion only
# 4. Normalise using training statistics only
mean = features[:num_train].mean(axis=0) #按 每一列 (每个特征) 求均值, 得到每个特征自己的均值;
std = features[:num_train].std(axis=0)

features = (features - mean) / std

# TODO: build supervised windows: X shape (samples, lookback, features), y shape (samples, 7)
# 5. Build supervised windows
# X: past 144 observations, y: next 7 temperatures
lookback = 144 #回看窗口大小, 用过去 144 个时间步作为一条输入样本;
horizon = 7 #预测步长, 一次性预测未来 7 个时间点的温度, 所以 y 的最后一维是 7;
batch_size = 256

features_tf = tf.convert_to_tensor(features, dtype=tf.float32) #把 numpy 数组转为 TensorFlow 张量, 方便 tf.data 构建数据集
temperature_tf = tf.convert_to_tensor(temperature, dtype=tf.float32)

def make_dataset(start, end, shuffle=False):
    max_start = end - lookback - horizon + 1 #能生成完整窗口的最大起始索引。
    ds = tf.data.Dataset.range(start, max_start) #生成一串起始索引号 i;

    if shuffle: #训练集才打乱窗口顺序; 验证、测试集不打乱, 保持时序;
        ds = ds.shuffle(10000, seed=42)

    def get_window(i):
        x = features_tf[i : i + lookback]
        y = temperature_tf[i + lookback : i + lookback + horizon]
        return x, y
    ds = ds.map(get_window, num_parallel_calls=tf.data.AUTOTUNE) #把每个索引号 i 映射成一组窗口 (x,y); AUTOTUNE让 TensorFlow 自动并行运算, 加速;
    ds = ds.batch(batch_size).prefetch(tf.data.AUTOTUNE) #.prefetch: 预加载下一批数据, GPU/CPU 空闲时提前准备, 减少等待
```

```

    return ds

train_ds = make_dataset(0, num_train, shuffle=True)
val_ds = make_dataset(num_train, num_train + num_val)
test_ds = make_dataset(num_train + num_val, n)

# 6. Naive baseline: repeat the last observed temperature
# 预测逻辑: 直接拿输入窗口最后一刻观测到的温度, 重复 7 次, 当作未来 7 步的预测。
def evaluate_naive(ds):
    total_abs_error = 0.0
    total_values = 0

    for x, y in ds:
        # 每个样本窗口里, 最后一个时间点的温度 (归一化后的值);
        # 反归一化! 把归一化的温度变回真实摄氏度; y 本身就是原始温度, 所以预测值必须还原, 不然 MAE 数值没有物理意义;
        last_temp = x[:, -1, temp_col] * std[temp_col] + mean[temp_col]
        # 把单个温度值复制 7 次, 得到预测 (batch, 7);
        pred = tf.repeat(last_temp[:, None], repeats=horizon, axis=1)

        # 整批所有预测点的绝对误差求和:
        total_abs_error += tf.reduce_sum(tf.abs(y - pred)).numpy()
        total_values += np.prod(y.shape)
    return total_abs_error / total_values # 总绝对误差 ÷ 总点数 = MAE (°C);

naive_mae = evaluate_naive(test_ds)
print(f"Naive baseline MAE: {naive_mae:.3f} °C")

# TODO: define, compile, and fit a model; evaluate MAE on a held-out set
# 7. Define and train model
model = keras.Sequential([
    layers.Input(shape=(lookback, features.shape[-1])),
    layers.Conv1D(32, 5, activation="relu"),
    layers.MaxPooling1D(2),
    layers.LSTM(32), # LSTM 循环层, 捕捉长时间依赖关系 (时序经典层), 输出一个向量
    layers.Dense(32, activation="relu"),
    layers.Dense(horizon),
])

model.compile(
    optimizer="rmsprop",
    loss="mae",
    metrics=["mae"],
)

callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_mae",
        patience=3,
        restore_best_weights=True,
    )
]

history = model.fit(
    train_ds,
    epochs=20,
    validation_data=val_ds,
    callbacks=callbacks,
)

# 8. Evaluate on held-out test set
results = model.evaluate(test_ds, return_dict=True)

metric = float(results["mae"])
print(f"Track A MAE (Moodle a5_metric): {metric:.3f}")
else:
    print("Skipping Track A")

```

```
(420551, 15)
Date Time p (mbar) T (degC) Tpot (K) Tdew (degC) rh (%) \
0 01.01.2009 00:10:00 996.52 -8.02 265.40 -8.90 93.3
1 01.01.2009 00:20:00 996.57 -8.41 265.01 -9.28 93.4
2 01.01.2009 00:30:00 996.53 -8.51 264.91 -9.31 93.9
3 01.01.2009 00:40:00 996.51 -8.31 265.12 -9.07 94.2
4 01.01.2009 00:50:00 996.51 -8.27 265.15 -9.04 94.1

VPmax (mbar) VPact (mbar) VPdef (mbar) sh (g/kg) H2OC (mmol/mol) \
0 3.33 3.11 0.22 1.94 3.12
1 3.23 3.02 0.21 1.89 3.03
2 3.21 3.01 0.20 1.88 3.02
3 3.26 3.07 0.19 1.92 3.08
4 3.27 3.08 0.19 1.92 3.09

rho (g/m**3) wv (m/s) max. wv (m/s) wd (deg)
0 1307.75 1.03 1.75 152.3
1 1309.80 0.72 1.50 136.1
2 1310.24 0.19 0.63 171.6
3 1309.19 0.34 0.50 198.0
4 1309.00 0.32 0.63 214.3
```

210275 105137 105139

Naive baseline MAE: 0.506 °C

Epoch 1/20

/Users/eva/Documents/Assignments/DATAX504/.venv/lib/python3.13/site-packages/keras/src/trainers/epoch_iterator.py:74: UserWarning: `shuffle=True` was passed, but will be ignored since the data `x` was provided as a tf.data.Dataset. The Dataset is expected to already be shuffled (via `.shuffle(buffer\_size)`).

```
self.data_adapter = data_adapters.get_data_adapter(
821/821 24s 29ms/step - loss: 1.5268 - mae: 1.5268 - val_loss: 0.5864 - val_mae: 0.5864
Epoch 2/20
821/821 24s 30ms/step - loss: 0.6344 - mae: 0.6344 - val_loss: 0.6333 - val_mae: 0.6333
Epoch 3/20
821/821 25s 31ms/step - loss: 0.5680 - mae: 0.5680 - val_loss: 0.4638 - val_mae: 0.4638
Epoch 4/20
821/821 26s 31ms/step - loss: 0.5340 - mae: 0.5340 - val_loss: 0.4465 - val_mae: 0.4465
Epoch 5/20
821/821 26s 32ms/step - loss: 0.5133 - mae: 0.5133 - val_loss: 0.5469 - val_mae: 0.5469
Epoch 6/20
821/821 26s 31ms/step - loss: 0.5001 - mae: 0.5001 - val_loss: 0.4706 - val_mae: 0.4706
Epoch 7/20
821/821 26s 32ms/step - loss: 0.4877 - mae: 0.4877 - val_loss: 0.4809 - val_mae: 0.4809
411/411 4s 9ms/step - loss: 0.4522 - mae: 0.4522
Track A MAE (Moodle a5_metric): 0.452
```

Track B — Text (AG News)

Four-class news topic classification.

You decide: how to load AG News (TFDS or another source), vocabulary / sequence length, Embedding + LSTM / Transformer / other, and the train/val/test protocol.

```
In [9]: if TRACK == "B":
import tensorflow as tf
import tensorflow_datasets as tfds

# TODO: load AG News (or a documented subset)
# 1. Load AG News
train_raw = tfds.load(
    "ag_news_subset",
    split="train[:90%]",
    as_supervised=True, #返回 (text, label), 而不是字典格式
)

val_raw = tfds.load(
    "ag_news_subset",
    split="train[90%:]",
```

```

        as_supervised=True,
    )

    test_raw = tfds.load(
        "ag_news_subset",
        split="test",
        as_supervised=True,
    )

    batch_size = 128

    train_ds = train_raw.batch(batch_size).prefetch(tf.data.AUTOTUNE)
    val_ds = val_raw.batch(batch_size).prefetch(tf.data.AUTOTUNE)
    test_ds = test_raw.batch(batch_size).prefetch(tf.data.AUTOTUNE)

    # TODO: text → integer sequences (TextVectorization, tokenizer, etc.)
    # 2. Text vectorisation 文本向量化层: 把英文单词映射成整数 ID
    max_tokens = 20000 #词表最多保留出现频率最高的 20000 个单词, 低频词统一变成一个特殊 OOV token
    sequence_length = 200

    vectorizer = layers.TextVectorization(
        max_tokens=max_tokens,
        output_mode="int", #输出整数序列
        output_sequence_length=sequence_length, #固定每条新闻输出长度 200: 长的截断, 短的补 0
    )

    train_text = train_raw.map(lambda text, label: text) #只提取训练集的文本, 丢掉标签
    vectorizer.adapt(train_text.batch(batch_size)) #只用训练文本统计词频、建立词表

    def vectorize_text(text, label):
        text = vectorizer(text)
        return text, label

    train_ds = train_ds.map(vectorize_text, num_parallel_calls=tf.data.AUTOTUNE) #.map 并行把三个数据集全部做文本编码。
    val_ds = val_ds.map(vectorize_text, num_parallel_calls=tf.data.AUTOTUNE)
    test_ds = test_ds.map(vectorize_text, num_parallel_calls=tf.data.AUTOTUNE)

    # TODO: define, compile, and fit a classifier for 4 classes
    # 3. Build sequence model
    model = keras.Sequential([
        layers.Input(shape=(sequence_length,), dtype="int64"),
        layers.Embedding(max_tokens, 128, mask_zero=True),
        layers.Bidirectional(layers.LSTM(64)),
        layers.Dropout(0.5),
        layers.Dense(64, activation="relu"),
        layers.Dense(4, activation="softmax"),
    ])
    model.compile(
        optimizer="rmsprop",
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )

    callbacks = [
        keras.callbacks.EarlyStopping(
            monitor="val_accuracy",
            patience=2,
            restore_best_weights=True,
        )
    ]
    history = model.fit(
        train_ds,
        epochs=10,
        validation_data=val_ds,
        callbacks=callbacks,
    )

    # TODO: report test accuracy as a percentage for Moodle a5_metric
    # 4. Evaluate on test set

```

```
results = model.evaluate(test_ds, return_dict=True)

metric = float(results["accuracy"] * 100)
print(f"Track B accuracy % (Moodle a5_metric): {metric:.2f}")
else:
    print("Skipping Track B")
```

WARNING:absl:Variant folder /Users/eva/tensorflow_datasets/ag_news_subset/1.0.0 has no dataset_info.json
/Users/eva/Documents/Assignments/DATAX504/.venv/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm

Downloading and preparing dataset Unknown size (download: Unknown size, generated: Unknown size, total: Unknown size) to /Users/eva/tensorflow_datasets/ag_news_subset/1.0.0...

```
DL Completed...: 0 url [00:00, ? url/s]
DL Size...: 0 MiB [00:00, ? MiB/s]

DL Completed...: 0%|          | 0/1 [00:00<?, ? url/s]
DL Size...: 0 MiB [00:00, ? MiB/s]

DL Completed...: 0%|          | 0/1 [00:03<?, ? url/s]
DL Size...: 0%|          | 0/11 [00:03<?, ? MiB/s]

Extraction completed...: 0 file [00:03, ? file/s]
DL Completed...: 0%|          | 0/1 [00:04<?, ? url/s]MiB]
DL Size...: 9%|█          | 1/11 [00:04<00:42, 4.25s/ MiB]

DL Completed...: 0%|          | 0/1 [00:04<?, ? url/s]
DL Size...: 18%|██         | 2/11 [00:04<00:38, 4.25s/ MiB]

DL Completed...: 0%|          | 0/1 [00:04<?, ? url/s]
DL Size...: 27%|███        | 3/11 [00:04<00:34, 4.25s/ MiB]

DL Completed...: 0%|          | 0/1 [00:04<?, ? url/s]
DL Size...: 36%|████       | 4/11 [00:04<00:29, 4.25s/ MiB]

DL Completed...: 0%|          | 0/1 [00:04<?, ? url/s]
DL Size...: 45%|█████      | 5/11 [00:04<00:25, 4.25s/ MiB]

DL Completed...: 0%|          | 0/1 [00:04<?, ? url/s]
DL Size...: 55%|██████     | 6/11 [00:04<00:21, 4.25s/ MiB]

DL Completed...: 0%|          | 0/1 [00:04<?, ? url/s]
DL Size...: 64%|███████    | 7/11 [00:04<00:17, 4.25s/ MiB]

DL Completed...: 0%|          | 0/1 [00:04<?, ? url/s]
DL Size...: 73%|████████   | 8/11 [00:04<00:12, 4.25s/ MiB]

DL Completed...: 0%|          | 0/1 [00:04<?, ? url/s]
DL Size...: 82%|█████████  | 9/11 [00:04<00:08, 4.25s/ MiB]

DL Completed...: 0%|          | 0/1 [00:04<?, ? url/s]
DL Size...: 91%|██████████ | 10/11 [00:04<00:04, 4.25s/ MiB]

DL Completed...: 0%|          | 0/1 [00:04<?, ? url/s]
DL Size...: 100%|██████████| 11/11 [00:04<00:00, 4.25s/ MiB]

DL Completed...: 100%|██████████| 1/1 [00:05<00:00, 5.00s/ url]
DL Size...: 100%|██████████| 11/11 [00:04<00:00, 4.25s/ MiB]

DL Completed...: 100%|██████████| 1/1 [00:05<00:00, 5.00s/ url]
DL Size...: 100%|██████████| 11/11 [00:05<00:00, 4.25s/ MiB]

DL Completed...: 100%|██████████| 1/1 [00:05<00:00, 5.00s/ url]]
DL Size...: 100%|██████████| 11/11 [00:05<00:00, 4.25s/ MiB]

DL Completed...: 100%|██████████| 1/1 [00:05<00:00, 5.00s/ url]]
DL Size...: 100%|██████████| 11/11 [00:05<00:00, 4.25s/ MiB]

DL Completed...: 100%|██████████| 1/1 [00:05<00:00, 5.00s/ url]]
DL Size...: 100%|██████████| 11/11 [00:05<00:00, 4.25s/ MiB]

Extraction completed...: 0%|          | 0/4 [00:05<?, ? file/s]

DL Completed...: 100%|██████████| 1/1 [00:05<00:00, 5.00s/ url]6s/ file]
DL Size...: 100%|██████████| 11/11 [00:05<00:00, 4.25s/ MiB]

DL Completed...: 100%|██████████| 1/1 [00:05<00:00, 5.00s/ url]6s/ file]
DL Size...: 100%|██████████| 11/11 [00:05<00:00, 4.25s/ MiB]

DL Completed...: 100%|██████████| 1/1 [00:05<00:00, 5.00s/ url]6s/ file]
DL Size...: 100%|██████████| 11/11 [00:05<00:00, 4.25s/ MiB]
```

```
DL Completed...: 100%|██████████| 1/1 [00:05<00:00, 5.00s/ url]6s/ file]
DL Size...: 100%|██████████| 11/11 [00:05<00:00, 4.25s/ MiB]

Extraction completed...: 100%|██████████| 4/4 [00:05<00:00, 1.27s/ file]
DL Size...: 100%|██████████| 11/11 [00:05<00:00, 2.17 MiB/s]
DL Completed...: 100%|██████████| 1/1 [00:05<00:00, 5.07s/ url]
Generating splits...: 0%|          | 0/2 [00:00<?, ? splits/s]
Generating train examples...: 0 examples [00:00, ? examples/s]
Generating train examples...: 66344 examples [00:01, 66343.59 examples/s]

Shuffling /Users/eva/tensorflow_datasets/ag_news_subset/incomplete.T91XFR_1.0.0/ag_news_subset-train.tfrecord-[0-9][0-9][0-9][0-9]-of-[0-9][0-9][0-9][0-9]...: 0%|          | 0/120000 [00:00<?, ? examples/s]
Generating splits...: 50%|██████    | 1/2 [00:01<00:01, 1.93s/ splits]
Generating test examples...: 0 examples [00:00, ? examples/s]

Shuffling /Users/eva/tensorflow_datasets/ag_news_subset/incomplete.T91XFR_1.0.0/ag_news_subset-test.tfrecord-[0-9][0-9][0-9][0-9]-of-[0-9][0-9][0-9][0-9]...: 0%|          | 0/7600 [00:00<?, ? examples/s]
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1790544289.192572 2120827 tf_record_dataset_op.cc:396] The default buffer size is 262144, which is overridden by the user specified `buffer_size` of 8388608
Dataset ag_news_subset downloaded and prepared to /Users/eva/tensorflow_datasets/ag_news_subset/1.0.0. Subsequent calls will reuse this data.
Epoch 1/10
/Users/eva/Documents/Assignments/DATAX504/.venv/lib/python3.13/site-packages/keras/src/trainers/epoch_iterator.py:74: UserWarning: `shuffle=True` was passed, but will be ignored since the data `x` was provided as a tf.data.Dataset. The Dataset is expected to already be shuffled (via `.shuffle(buffer_size)`).
  self.data_adapter = data_adapters.get_data_adapter(
844/844 ██████████ 260s 307ms/step - accuracy: 0.8352 - loss: 0.4457 - val_accuracy: 0.8989 - val_loss: 0.3043
Epoch 2/10
844/844 ██████████ 242s 286ms/step - accuracy: 0.9137 - loss: 0.2644 - val_accuracy: 0.9005 - val_loss: 0.2980
Epoch 3/10
844/844 ██████████ 249s 295ms/step - accuracy: 0.9256 - loss: 0.2281 - val_accuracy: 0.9031 - val_loss: 0.2928
Epoch 4/10
844/844 ██████████ 241s 285ms/step - accuracy: 0.9342 - loss: 0.2011 - val_accuracy: 0.9022 - val_loss: 0.2978
Epoch 5/10
844/844 ██████████ 239s 283ms/step - accuracy: 0.9407 - loss: 0.1784 - val_accuracy: 0.8977 - val_loss: 0.3165
60/60 ██████████ 4s 74ms/step - accuracy: 0.9003 - loss: 0.2896
Track B accuracy % (Moodle a5_metric): 90.03
```

Written responses

Architecture summary (Moodle a5_architecture , max 100 words)

Layers, sizes, and why this design fits your track.

Input preparation (Moodle a5_tokenisation , max 100 words)

*Windowing / normalisation **or** tokenisation / padding / masking — whichever applies.*

Your answers here.